Funciones virtuales y override

Una llamada virtual usa el tipo real del objeto

C++20 • Polimorfismo a través de un puntero o una referencia a la base.

Una interfaz abstracta

La clase base declara una función virtual pura. No se puede instanciar.

```
class Shape {
public:
    virtual ~Shape() = default;
    virtual double area() const = 0;
};
```

Una implementación concreta

La clase derivada implementa la función virtual y usa override.

```
class Circle : public Shape {
    double radio;
public:
    explicit Circle(double r) : radio(r) {}
    double area() const override {
        return std::numbers::pi * radio * radio;
    }
};
```

Sigue la llamada

Se hace la llamada a través de un puntero a la base. Se ejecuta la función del tipo real del objeto.

```
Circle c{5};
Shape* figura = &c;
std::cout << std::fixed
          << std::setprecision(2)
          << figura->area();
```

¿Qué ocurre?

La llamada virtual usa el tipo real del objeto apuntado.



Salida (área con dos decimales):

```
78.54
```

Qué aporta cada palabra

`virtual`	habilita el despacho dinámico.
`= 0`	declara una función virtual pura.
`Shape`	es abstracta y no se puede instanciar.
`override`	exige que la función redefina una virtual compatible de la base.

La firma importa

Si omites `const` en `Circle::area()`, no redefinirá esa función `const` de Shape. Con `override`, el compilador detecta el error.

```
class Circle : public Shape {
public:
    double area() override {  // Error: falta const
        return 0.0;
    }
};
```

Destrucción polimórfica

Si eliminas un objeto derivado mediante un puntero a la base, la base necesita un destructor virtual.
Pasar o copiar un derivado por valor como Shape puede perder su parte derivada cuando la base es concreta: ese problema se llama *slicing*.

C++20: [class.virtual], [class.abstract], [expr.delete].